mattpocock/skills / はじめての人向けガイド

AI エージェントの 39 のスキルを、全部わかる

全体像 → はじめかた → 1 つずつの紹介。どれが自分に合うかが分かり、すぐ使える形で。

github.com/mattpocock/skills（MIT・★31 万）／ 2026-11-05 時点

3 行でいうと、これ

01

AI エージェントのための「小さな規律集」

BMAD ・ Spec-Kit のようにプロセスを丸ごと所有する道具ではなく、書き換えやすく組み合わせられる断片の集まり。

02

39 個あるが、芯は「アイデア → 出荷」の 1 本の流れ

問い詰め → 仕様 → チケット → 実装 → レビュー。ほかは横から入るか、足元で語彙を支える。

03

いちばん効くのは、たぶん「問い詰め」と「共有語彙」

作者自身が、この 2 つが一番クールだと言っている。コードを書く前のズレを、ここで潰す。

なぜ作られたのか：４つの失敗モード

README は「よくある失敗」と「効くスキル」を 1 対 1 で対応させている

①	**意図と違うものを作る** よくある失敗	作る前に、容赦なく問い詰められる 対策	grill-me / grill-with-docs
②	**説明が冗長で、コードも読みにくい** よくある失敗	共有語彙（GLOSSARY）と ADR を作る 対策	domain-modeling / grill-with-docs
③	**コードが動かない** よくある失敗	フィードバックループを回す 対策	tdd / diagnosing-bugs
④	**泥団子（ball of mud）になる** よくある失敗	深いモジュールの設計を毎日少しずつ 対策	codebase-design / improve-codebase-architecture

39 のスキルは、４つの箱に入っている

まずは箱の名前と数を覚えれば、全体の 8 割はつかめたことになる

engineering	20 skills	コード作業の日常道具。レビュー・ TDD ・設計・チケット・リリース
productivity	7 skills	コードに依存しない思考ツール。問い詰め・引き継ぎ・教育
in-progress	8 skills	開発中。挙動が変わる可能性あり（書く・采配する系が多め）
misc	4 skills	凍結中。メンテされない（ git の安全策・ pre-commit など）

※ ほかに deprecated （中身は README のみ）がある

軸はたった 1 本：誰が呼べるか

この区別が分かると、このリポジトリの設計がほぼ分かる

user-invoked

自分で打つ

人間が `/name` と打ったときだけ動く。

- 仕事は「段取り（オーケストレーション）」
- モデルからは絶対に呼べない
- 説明文は人間向け（短い 1 行）

model-invoked

自動でも呼ばれる

人間からも、モデルが「これだ」と選んでも動く。

- 仕事は「再利用できる規律」
- 「〜したい」の言い回しで自動発火する
- 説明文はモデル向け（きっかけの言葉入り）

守っている不変条件

user-invoked は model-invoked を呼べるが、user-invoked 同士は呼べない。

→ コマンドが勝手に別のコマンドを連鎖して暴走する事故を、構造で防いでいる。

流れは 1 本。そこに 3 つの横道が合流する

「アイデア → 出荷」の本流。ほかのスキルは、この流れを支えるか、横から入る。

1. **grill-with-docs** — 問い詰めて、用語集と ADR を育てながら、作りたいものを固める
2. **to-spec** — 会話を仕様書にして issue tracker に置く（聞き直さない）
3. **to-tickets** — 仕様を、依存関係を宣言したチケット群に割る
4. **implement** — 実装する。内部で /tdd を回し、最後に /code-review 。仕様全体なら /implement-spec が並列で建てる
5. **code-review → pr** — 2 軸レビューで締め、 PR 本文を整える

横から合流する 3 つ

/triage

外から来た issue を整えて、実装できる形にする

/diagnosing-bugs

壊れたものから入る。直したら retro へ

/wayfinder

霧の中の巨大な仕事を、決定の地図にする

足元で支える語彙

domain-modeling ／ codebase-design が、上の全部の下で効く

コンテキストの使い方が、この流れの隠れた主役

「どのスキルを使うか」と同じくらい、「どの窓で回すか」が効く

1 〜 3 は、 1 つの窓で切らない

問い詰め・仕様・チケットは、同じ思考の続き。 /to-tickets まで /clear も /compact もしない。

各 /implement は、チケットから始めればいいので毎回まっさらでよい。

スマートゾーン（約 150k トークン）

その中でなら、モデルはまだ鋭く考えられる。

超えそうなら、劣化したまま押さず、区切りのいい所で /compact 。

区切りでの 5 つの選択肢

続ける

コストも損失もゼロ。まずこれを検討

/clear

ここでの内容が次に要らないとき

/handoff

別のハーネス・別のディレクトリ・同僚へ渡す（可搬）

サブエージェント

狭い作業を別の窓に投げて報告だけ受ける

/compact

既定。木の底の選択肢で、最初に選ぶものではない

詳しくは skills/engineering/ask-matt/PHASE-BOUNDARIES.md （5 つの質問の決定木）。

入れる：使っているエージェントに 1 つだけ

プラグイン（自動更新）か、コピー（手で編集・手で更新）かを選ぶ

Claude Code

自動で更新される

```
claude plugin install mattpocock-skills@claude-plugins-official
```

Codex

起動時に更新される

```
codex plugin marketplace add mattpocock/skills
codex plugin add mattpocock-skills@mattpocock
```

GitHub Copilot

CLI と VS Code の両方

```
copilot plugin marketplace add mattpocock/skills
copilot plugin install mattpocock-skills@mattpocock
```

その他のエージェント

cursor ・ opencode ・ devin など

```
npx skills@latest add mattpocock/skills -a <agent>
```

※ 両方入れると全スキルが二重に入る。 1 つのエージェントにだけ。

最初の 1 回だけ： /setup-matt-pocock-skills

ほかの engineering スキルは、これが済んでいる前提で動く

1. **issue tracker は何か** — GitHub Issues / GitLab / ローカルの Markdown / その他（自由記述）
2. **トリアージで使うラベルは何か** — needs-triage や ready-for-afk を、実際のラベル名に結びつける
3. **作った docs をどこに置くか** — GLOSSARY.md や ADR の置き場を決める

自分で作った issue tracker でもいい。
設定はリポジトリごとに 1 回。別のリポジトリで使うなら、そこでまた走らせる。

迷ったら： /ask-matt に「この状況なら何を使う？」と聞く。
ただし ask-matt の要約は案内用。実行前に、そのスキルの SKILL.md を読むのが前提。

今日やるなら、この３手

全部入れる必要はない。まず 1 本、通してみる

手 1

/grill-with-docs

次の変更について、容赦なく聞かれる

答えるうちに GLOSSARY.md と ADR が勝手に育つ。ここが一番効く。

手 2

/implement

固まったところで実装

内部で /tdd を回し、最後に /code-review が走る。チケット 1 枚ずつでよい。

手 3

/retro

セッションの後に、環境を見直す

コードではなく「次に同じミスをしない仕組み」を提案してくれる。

順番が大事：先に問い詰め、次に実装、最後に振り返り。いきなり実装から入ると、ズレたものが出てくる。

困りごとから、引く

「今どれを使えばいいか」で迷ったら、この表から

作りたいものが曖昧	/grill-with-docs / grill-me
仕様にしたい	/to-spec
チケットに割りたい	/to-tickets
実装したい	/implement / implement-spec
バグが直らない	/diagnosing-bugs
レビューしたい	/code-review
PR を出したい	/pr
コードが汚い	/improve-codebase-architecture
用語がバラバラ	/domain-modeling
巨大で霧の中	/wayfinder
手順を人に任せたい	/wizard
セッションを引き継ぐ	/handoff
教えてほしい	/teach
環境を整えたい	/setup-pre-commit / git-guardrails

※ どれか分からなければ /ask-matt 。状況を話せば、使う順番まで返してくれる。

20
engineering

コード作業の日常道具。この箱だけで「アイデア → 出荷」が一周できる。

自分で打つものが 11 、自動でも呼ばれるものが 9 。前半の 11 が「段取り役」、後半の 9 が「規律」だと考えると分かりやすい。

/setup-matt-pocock-skills
/ask-matt
/grill-with-docs
/triage
/to-spec
/to-tickets
/implement
/implement-spec
/wayfinder
/improve-codebase-architecture
/retro
/tdd
/code-review
/diagnosing-bugs
/prototype
/research
/codebase-design
/domain-modeling
/pr
/wizard

自分で打つ　モデルは呼べない

/setup-matt-pocock-skills

何をするか

リポジトリを、他のスキルが動く状態に整える。 issue tracker ・トリアージのラベル・ docs の置き場を決める。

いつ使うか

- このリポジトリで初めて engineering スキルを使うとき
- 別のリポジトリに移ったとき（リポジトリごとに 1 回）

ひとこと

最初に 1 回だけ。ほかの engineering スキルは「もう設定済み」を前提に走るので、飛ばすと途中で止まる。

自分で打つ　モデルは呼べない

/ask-matt

何をするか

「この状況なら、どのスキル？」を答えるルーター。本流・横道・単発のどこにいるかを教えてくれる。

いつ使うか

- どのスキルを使うか忘れたとき
- この全体像をもう一度たどりたいとき

ひとこと

要約は案内用で、定義ではない。実行前に、そのスキルの SKILL.md 本体を読ませるのが前提。

自分で打つ　モデルは呼べない

/grill-with-docs

何をするか

容赦なく問い詰められる。答えるうちに GLOSSARY.md と ADR が、その場で育っていく。

いつ使うか

- リポジトリで作業していて、変更を始める前
- 言葉が曖昧なまま作り始めそうなとき

ひとこと

作者いちばんの人気スキル。作業ディレクトリが無いなら /grill-me（同じ中身で、記録を残さない）。

自分で打つ　モデルは呼べない

/triage

何をするか

外から来た issue と PR を、状態機械（needs-triage → ready-for-afk など）で流し、エージェントが拾える形にする。

いつ使うか

- バグ報告や要望が溜まってきたとき
- 他人の PR を、まず仕分けたいとき

ひとこと

自分で作ったチケット（/to-tickets 産）には使わない。すでに実装可能な形になっている。

自分で打つ　モデルは呼べない

/to-spec

何をするか

いまの会話を仕様書にして、 issue tracker に公開する。インタビューはせず、話したことを束ねるだけ。

いつ使うか

- 複数セッションにまたがる作業に入る前
- 問い詰めが終わって、形にしたいとき

ひとこと

聞き直さないのが持ち味。だから /grill-with-docs の直後に使うのが一番効く。

自分で打つ　モデルは呼べない

/to-tickets

何をするか

仕様を「トラサーバレット」なチケットに割る。各チケットが、自分を止めている依存を宣言する。

いつ使うか

- 仕様を、実装できる粒度に分けたいとき
- 手分けや並列化をしたいとき

ひとこと

チケットは手順書ではなくタスクグラフ。依存が解けた「前線」から着手する、という考え方。

自分で打つ　モデルは呼べない

/implement

何をするか

仕様やチケット 1 つを実装する。合意した継ぎ目で /tdd を回し、最後に /code-review を走らせる。

いつ使うか

- チケット 1 枚を、その場で片付けたいとき
- 1 つの窓で、仕様から実装まで通したいとき

ひとこと

チケットごとに /clear して文脈を捨てる前提。チケットが自己完結しているから成立する。

自分で打つ　モデルは呼べない

/implement-spec

何をするか

仕様全体を 1 つの統合ブランチで実装する。タスクグラフの準備できた前線に、実装サブエージェントを並列投入。

いつ使うか

- 1 枚ずつ自分で回すより、まとめて任せたいとき
- チケットが多く、並列に進められる形になっているとき

ひとこと

サブエージェントは毎回まっさらな文脈で始める。連絡は文脈ポインタ（仕様・チケット・コミット）で疎に。

自分で打つ　モデルは呼べない

/wayfinder

何をするか

1 セッションに収まらない、霧の中の仕事を、決定チケットの「地図」にする。 1 つずつ解いて、道を晴らす。

いつ使うか

- 新規プロジェクトや巨大な機能で、道筋が見えないとき
- 何から決めるべきか自体が分からないとき

ひとこと

計画であって実行ではない。地図が晴れたら /to-spec に合流する。小さく切れる仕事には使わない。

自分で打つ　モデルは呼べない

/improve-codebase-architecture

何をするか

コードベースを走査して「深掘りできる候補」を HTML レポートで出す。選んだ候補を、そのまま問い詰める。

いつ使うか

- 数日に 1 回、余裕のあるときに
- エージェントが迷いやすいコードになってきたとき

ひとこと

「救助」ではなく「調査」。古い泥団子も、実際に候補は出るが、解いてはくれない。

自分で打つ　モデルは呼べない

/retro

何をするか

セッションを振り返り、コードではなく「環境」の改善を重い順に提案する。探索性・自動チェック・規約・道具。

いつ使うか

- ビルドの後、特にうまくいかなかった後
- /clear する前の、同じセッションの中で

ひとこと

機械的なミスは決定的なチェックに、判断が要るものは規約に落とす。次が良い環境から始まる。

自動でも呼ばれる

/tdd

何をするか

red-green-refactor 。失敗するテストを先に書き、通してから整える。縦に 1 スライスずつ進む。

いつ使うか

- 振る舞いを、テストから作りたいとき
- 不具合を、再発しない形で直したいとき

ひとこと

単体でも使えるが、 /implement が継ぎ目ごとに内部で呼ぶ。良いテストと悪いテストの指針も持っている。

自動でも呼ばれる

/code-review

何をするか

定点（コミット・ブランチ・タグ・ merge-base ）からの差分を、 2 軸でレビューする。規約（ Standards ）と仕様（ Spec ）。

いつ使うか

- ブランチや PR を見るとき
- 作業中の差分を、出す前に見たいとき

ひとこと

2 軸を別のサブエージェントに走らせて並べる。混ぜると、片方の観点が必ず消えるから。

自動でも呼ばれる

/diagnosing-bugs

何をするか

難しいバグの診断ループ。赤くなるフィードバックループを作る → 最小化 → 仮説 → 計測 → 修正 → 回帰テスト。

いつ使うか

- 一目では分からないバグ
- 再現しない不具合・間欠的な失敗・性能劣化

ひとこと

ループができるまで理論を立てない、が芯。締めは「何が防げたか」を /retro で問う。

自動でも呼ばれる

/prototype

何をするか

使い捨ての試作で、設計の問いに答える。状態やロジックは HTML 1 枚、 UI は複数案を 1 ルートで切り替え。

いつ使うか

- 紙の上では決められない問いがあるとき
- この状態モデルで本当に回るか、見てみたいとき

ひとこと

「使い捨て」は書き方の制約。答えは本体に畳まれ、試作は一次資料としてブランチに残す。

自動でも呼ばれる

/research

何をするか

一次資料に当たって調べ、引用つきの Markdown をリポジトリに残す。裏のエージェントとして走る。

いつ使うか

- 調べ物を、作業と並行で進めたいとき
- ドキュメントや API の事実を、確かな出どころで押さえたいとき

ひとこと

読みの足仕事を委譲する。成果物は /grill-with-docs に持ち込む。調べが思考の代わりにはならない。

自動でも呼ばれる

/codebase-design

何をするか

深いモジュールの語彙を提供する。 module / interface / depth / seam / adapter / leverage / locality 。

いつ使うか

- モジュールの形を決めるとき
- テストしやすさ・ AI の探索しやすさを上げたいとき

ひとこと

手順ではなく語彙の層。 /tdd と /improve-codebase-architecture が内部で使う。

自動でも呼ばれる

/domain-modeling

何をするか

ドメインの言葉を削る。用語を疑い、 edge case で叩き、 GLOSSARY と ADR をその場で更新する。

いつ使うか

- 言葉が曖昧なとき
- 1 つの語が 3 つの意味で使われているとき

ひとこと

これも語彙の層。 /grill-with-docs が駆動する。読むだけではこのスキルではなく、ただの参照。

自動でも呼ばれる

/pr

何をするか

PR 本文の型。最小の視覚で要約し、前後の証拠を添え、一方通行か両方向ドアかと爆発半径を書く。

いつ使うか

- PR を書くとき（モデルが自動で使う）
- PR 本文が薄くなりがちなとき

ひとこと

「戻せるか」を必ず書くのが芯。レビューする人が、どこまで慎重になるべきか決められる。

自動でも呼ばれる

/wizard

何をするか

人間にしかできない手順を、対話式の bash にする。URL を開き、値を捕まえ、.env とシークレットに書く。

いつ使うか

- インフラの用意・資格情報・CI secret
- 見慣れない外部ダッシュボード・一度きりの移行

ひとこと

エージェントが自分でできる作業には使わない。人が本当にループに入る所だけ。

07
productivity

コードに依存しない思考ツール。問い詰め・引き継ぎ・教育。engineering と組み合わせて使う。

7 つのうち、自分で打つのは 6 つ。grilling だけが「原関数」で、ふだんは他のスキル越しに触れることになる。

/grilling

/grill-me

/handoff

/teach

/to-questionnaire

/wait-what

/writing-for-agents

自動でも呼ばれる

/grilling

何をするか

問い詰めの「原関数」。ラウンド制で質問を返し、事実はエージェントが調べ、決定はあなたがする。

いつ使うか

- 他のスキルが内部で使う、という理解でよい
- ラッパーなしの問い詰めを、そのまま使いたいとき

ひとこと

grill-me ・ grill-with-docs ・ triage ・ wayfinder ・ improve-… は、全部この上に乗っている。

自分で打つ　モデルは呼べない

/grill-me

何をするか

容赦ない問い詰め。ただし状態を持たない。ファイルも作らなければ、 GLOSSARY も作らない。

いつ使うか

- 作業ディレクトリが無いとき（企画・設計・文章）
- リポジトリの外で考えを詰めたいとき

ひとこと

リポジトリの中なら /grill-with-docs の方が良い。同じ中身で、記録が残るから。

自分で打つ　モデルは呼べない

/handoff

何をするか

いまの会話を、持ち運べる Markdown に圧縮する。別のエージェントが続きを拾える形に。

いつ使うか

- 別のハーネス・別のディレクトリ・同僚に渡すとき
- 試作を別ディレクトリで詰めたいとき（往復に使う）

ひとこと

使いどころは狭い。可搬性を買う道具で、単なる文脈の掃除なら /compact で足りる。

自分で打つ　モデルは呼べない

/teach

何をするか

複数セッションかけて、概念やスキルを教える。今のディレクトリを、学習の状態として使う。

いつ使うか

- 新しい技術を、体系的に身につけたいとき
- 学んだことを、記録しながら進めたいとき

ひとこと

ミッション・学習記録・用語集・教材の 4 つの書式を持つ、ちょっとした学習管理システム。

自分で打つ　モデルは呼べない

/to-questionnaire

何をするか

自分では答えられない判断を、答えられる人への質問票にする。宛先を問い詰めて、質問を狙う。

いつ使うか

- 必要な情報が、自分の頭にもコードにも無いとき
- 関係者に、非同期で聞きたいとき

ひとこと

/grill-me の逆。あなたを問い詰めるのではなく、「送り方」を問い詰める。返事は仕様の材料になる。

自分で打つ　モデルは呼べない

/wait-what

何をするか

届かなかったメッセージを、言い直させる。足りない文脈を補い、 GLOSSARY の語彙で、平易に。

いつ使うか

- エージェントの説明が、頭に入ってこなかったとき
- 話が噛み合っていないと感じた、その場で

ひとこと

事後の修正。事前の薬は /grill-with-docs で、早めに言葉を揃えておくこと。

自動でも呼ばれる

/writing-for-agents

何をするか

エージェントが読む文書の書き方。スキル・ AGENTS.md ・ポインタで指す資料の作法。

いつ使うか

- スキルを書く・直すとき
- AGENTS.md / CLAUDE.md を触るとき

ひとこと

「呼ばれる言い回し」と「読まれる構造」の両方を扱う。長すぎる指示文の整理にも効く。

08
in-progress

開発中。挙動は変わりうる。「書く」「采配する」系の実験が多く、当たりを引けば強い。

実験中なので、業務の本流に置くのは避けた方が無難。試すのは、失敗しても痛くない作業から。

/chief-of-staff

/claude-handoff

/loop-me

/software-factory

/setup-ts-deep-modules

/writing-fragments

/writing-shape

/writing-beats

自分で打つ　モデルは呼べない

/chief-of-staff

何をするか

長期の目標を 1 セッションで追う。サブエージェントとスケジュールを采配する、参謀役。

いつ使うか

- 何日もかかる目標を、 1 つの窓で回したいとき
- 並行して走る作業を、まとめて采配したいとき

ひとこと

「目先のタスク」と「次のタスクが楽になる環境」の 2 トラックで考える、と指示されている。

自分で打つ　モデルは呼べない

/claude-handoff

何をするか

いまの会話を、新しいバックグラウンドエージェントに渡す。要約を書いて、そのまま起動する。

いつ使うか

- 作業を止めずに、別のエージェントへ続きをやらせたいとき

ひとこと

claude --bg を使う前提。要約には「次に呼ぶべきスキル」も書かれる。

自分で打つ　モデルは呼べない

/loop-me

何をするか

人生の「ループ」（毎週・毎朝・繰り返す活動）を見つけ、それを workflow 仕様にする問い詰め。

いつ使うか

- 繰り返しの作業を、委譲できる形にしたいとき

ひとこと

workflows/*.md が正。ループは入れ子になっている、という見方で探す。

自分で打つ　モデルは呼べない

/software-factory

何をするか

チケット・ PR ・スケジュールをマージ済みコードに変える「工場」を設計するための共通言語。

いつ使うか

- エージェント群で開発を回す仕組みを、言葉から設計したいとき

ひとこと

用語集。 task ・ run などの語を固定し、避ける語を決めて会話を揃える。

自分で打つ　モデルは呼べない

/setup-ts-deep-modules

何をするか

TypeScript リポジトリに dependency-cruiser を入れ、各パッケージを深いモジュールにする。

いつ使うか

- TS で、内部を隠したパッケージ構成にしたいとき

ひとこと

入口ファイル経由でしか辿れない形を、機械的に強制する。

自分で打つ　モデルは呼べない

/writing-fragments

何をするか

書く・探索。断片を掘る。構造はまだ決めない。断片は 1 つの Markdown に足されていく。

いつ使うか

- 何か書きたいが、形が見えていないとき

ひとこと

ここでは絶対に構成を決めない。決めるのは writing-shape の仕事。

自分で打つ　モデルは呼べない

/writing-shape

何をするか

書く・収穫。素材の山を、記事に整形する。段落ごとに進む。素材ファイルは読み取り専用。

いつ使うか

- 素材はある。記事にしたいとき

ひとこと

素材を編集しないのが約束。整形と素材を混ぜると、どちらも信用できなくなる。

自分で打つ　モデルは呼べない

/writing-beats

何をするか

書く・収穫。素材を「ビート」の連なりとして組み、各ビートが寄りかかる前に用語を固める。

いつ使うか

- 素材を、読ませる流れに組みたいとき
- 説明の順番で悩んでいるとき

ひとこと

choose-your-own-adventure 形式で、 1 ビートずつ選びながら進む。

misc ：凍結中（メンテされない）

4 つあるが、作者は「凍結・未メンテ」と明言している。参考までに。

/git-guardrails-claude-code	危険な git コマンド（ push ・ reset --hard ・ clean ・ branch -D など）を、実行前にフックで止める仕組みを入れる。
/setup-pre-commit	Husky と lint-staged を入れ、コミット時に整形・型チェック・テストを走らせる。
/migrate-to-shoehorn	テストの `as` による型アサーションを、 @total-typescript/shoehorn に置き換える。
/scaffold-exercises	教材のディレクトリ構造（節・問題・解答・解説）を作り、 lint が通る状態にする。

入れても害はないが、直されない前提で。 git の安全策だけは、自分で書いても同じものが作れる。

どれから始めるか

全部を入れる必要はない。 1 つ選んで、 1 週間だけ使ってみるのが早い。

まず 1 本、通してみたい この 3 つで「アイデア → 出荷」の芯が通る	grill-with-docs → implement → code-review
品質を上げたい フィードバックの速さが、そのまま品質になる	tdd ／ diagnosing-bugs ／ code-review
大きな仕事を任せたい 霧を晴らしてから、並列で建てる	wayfinder → to-spec → to-tickets → implement-spec
言葉を揃えたい 用語集が育つと、以後の全セッションが短くなる	grill-with-docs ／ domain-modeling
人のコードに入る 外から来たものを整え、環境を直して次に備える	triage ／ code-review ／ retro

出典： github.com/mattpocock/skills （ MIT License, ★31 万 , 2026-11-05 時点）／ 内容は各 SKILL.md と README 、 author の解説にもとづく
※ misc は凍結中、 in-progress は実験中。挙動は変わりうる。